

Informe Técnico: Compilador DSL para Máquina de Ritmos

Pablo Germano, Federico Klobberdanz, Valentin Ontivero, Mauro
Vella

ITBA

Noviembre 2024

1. Introducción	2
Objetivo del Proyecto	2
2. Modelo Computacional	2
2.1. Dominio	3
Conceptos del Dominio	3
Restricciones del Dominio	4
2.2. Lenguaje	4
Características del Lenguaje	4
Gramática Formal	5
3. Implementación	9
3.1. Frontend	9
3.2. Backend	10
3.3. Adicionales	15
3.4. Dificultades Encontradas	18
4. Futuras Extensiones	21
4.1 Extensiones del lenguaje	21
4.2 Extensiones de análisis	21
4.3 Extensiones del backend	21
4.4 Herramientas y performance	21
5. Conclusiones	22
5.1. Lecciones Aprendidas	22
5.2. Reflexión Final	22
6. Referencias	24
7. Bibliografía	25

1. Introducción

El presente informe documenta el diseño, implementación y validación de un compilador completo para un Lenguaje Específico de Dominio (DSL) orientado a la creación de patrones rítmicos para máquinas de ritmos y 'drum machines'. El lenguaje del proyecto busca simplificar la composición musical mediante un lenguaje declarativo de alto nivel para generar archivos MIDI.

Objetivo del Proyecto

Implementar un compilador completo que incluya todas las fases clásicas de compilación: análisis léxico, análisis sintáctico, análisis semántico, y generación de código MIDI.

Y diseñar un DSL específico para esta tarea que capture los conceptos fundamentales de la composición rítmica: tempo, compases, subdivisiones, patrones de ritmo, y asignación instrumental.

Garantizar corrección sin conflictos shift/reduce en el parser, cobertura completa de pruebas, y ausencia de fugas de memoria.

Generar salida en formato MIDI estándar, reproducible en cualquier reproductor MIDI sin dependencias adicionales. Además se añadió la posibilidad de generar archivos FLAC/WAV/MP3 como salida ya que es más fácil su reproducción.

2. Modelo Computacional

2.1. Dominio

El dominio del compilador es la composición algorítmica de patrones rítmicos para instrumentos de percusión y melódicos en el contexto de producción musical.

Conceptos del Dominio

El modelo computacional se fundamenta en los siguientes conceptos musicales:

- **Tempo:** Velocidad de reproducción medida en pulsaciones por minuto (BPM). Define la base temporal de toda la composición.
- **Compás:** Unidad estructural de organización temporal. Un compás agrupa un número determinado de pasos.
- **Pasos (Steps):** Subdivisiones dentro de un compás. Representan la resolución temporal mínima del lenguaje.
- **Patrón de Ritmo:** Secuencia de eventos musicales que pueden ser:

HIT ("x"): Golpe de percusión o evento activo.

SILENCE("."): Silencio o ausencia de evento.

NOTE: Nota melódica con altura definida (ej. "E2", "A#2", "Bb3").

- **Operadores sobre Ritmos:**

Repetición ("* N"): Duplica un patrón N veces consecutivas.

Concatenación ("+"): Combina dos patrones secuencialmente.

- **Instrumentos:** Entidad que ejecuta un patrón específico durante rangos de compases definidos (rangos activos).
- **Rangos Activos:** Especificación de en qué compases un instrumento debe reproducirse. Soporta definición por intervalos ("1-4") y concatenación de rangos discontinuos ("1-4 + 6-8 + 10-16").

Ejemplo:

```

tempo 140
compasses 16
steps 8

pattern kickPattern {
    rhythm [x,.,.,.,x,.,.,.]
}

instruments {
    kick {
        pattern kickPattern
        active 1-16
    }
}

```

Este programa especifica una estructura de 16 compases de 8 pasos cada uno, donde todos los compases están activos, a una velocidad de 140 BPM. El patrón es solo un kick drum en los pasos 1 y 5 de cada compás.

Restricciones del Dominio

- Los identificadores de patrones e instrumentos deben ser únicos dentro de su ámbito.
- Los instrumentos sólo pueden referenciar patrones previamente definidos.
- Los rangos activos deben satisfacer que el inicio sea menor que el fin, y que el fin sea menor que la cantidad de compases total declarada.
- Las notas melódicas deben seguir la notación anglosajona: `[A-G][#b]?[0-9]`
- Tempo, compases y pasos deben ser enteros estrictamente positivos.

2.2. Lenguaje

Características del Lenguaje

El DSL para máquina de ritmos es un lenguaje declarativo, imperativo de bajo nivel semántico, y de propósito específico con las siguientes propiedades:

- Declarativo: El programador especifica qué debe sonar, no cómo generarlo. El compilador maneja los detalles de timing MIDI, cálculo de delta times, codificación VLQ, etc.
- Tipado estático implícito: Los tipos se infieren del contexto sintáctico. No hay declaraciones explícitas de tipos.
- Ámbito global simple: Los patrones tienen ámbito global. No hay anidamiento de scopes ni shadowing.
- Modular: Soporta importación de archivos externos mediante la directiva “remember”.

Gramática Formal

El lenguaje está definido por una gramática libre de contexto (Tipo 2 en la jerarquía de Chomsky) implementada con un parser LALR(1) sin conflictos.

Gramática BNF Completa

```

<program> ::= <imports> <declarations> <patterns> <instruments>

<imports> ::= ε
           | <import_list>

<import_list> ::= <import>
               | <import_list> <import>

<import> ::= "remember" STRING_LITERAL

<declarations> ::= "tempo" INTEGER
                 | "tempo" INTEGER "compasses" INTEGER
                 | "tempo" INTEGER "compasses" INTEGER "steps" INTEGER

<patterns> ::= ε
           | <pattern_list>

<pattern_list> ::= <pattern_def>
                 | <pattern_list> <pattern_def>

<pattern_def> ::= "pattern" ID "{" "rhythm" <rhythm_expr> "}"

<rhythm_expr> ::= <rhythm_array>
               | <rhythm_expr> "+" <rhythm_expr>
               | <rhythm_array> "*" INTEGER

<rhythm_array> ::= "[" <rhythm_element_list> "]"

```

```

<rhythm_element_list> ::= <rhythm_element>
                        | <rhythm_element_list> "," <rhythm_element>

<rhythm_element> ::= "x"
                  | "."
                  | NOTE

<instruments> ::= "instruments" "{" <instrument_list> "}"

<instrument_list> ::= <instrument_def>
                    | <instrument_list> <instrument_def>

<instrument_def> ::= ID "{" "pattern" ID "active" <active_range_list> "}"

<active_range_list> ::= <active_range>
                      | <active_range_list> "+" <active_range>

<active_range> ::= INTEGER "-" INTEGER

```

Precedencia y Asociatividad de Operadores

El lenguaje define dos operadores binarios sobre expresiones rítmicas:

La repetición que se denotará con un asterisco ("*"), precedencia alta de izquierda. Y la concatenación que se denota con un más ("+"), de precedencia baja de izquierda.

Ejemplo de evaluación:

```

rhythm [x,.] * 2 + [.,x]
Equivale a: ([x,.] * 2) + [.,x]
Resultado: [x,.,x,.] + [.,x] = [x,.,x,., .,x]

```

Esta precedencia garantiza que "*" se evalúe antes que "+", consistente con la intuición matemática.

Ejemplo Completo Paso a Paso

```

/* === SECCIÓN 1: IMPORTACIONES === */
/* Permite reutilizar definiciones de patrones de bibliotecas externas
*/

```

```

remember "lib/kick.dsl"
remember "lib/snare.dsl"
remember "lib/hihat.dsl"

/* === SECCIÓN 2: DECLARACIONES GLOBALES === */
tempo 140          /* Velocidad: 140 pulsaciones por minuto */
compasses 16       /* Estructura: 16 compases */
steps 8            /* Resolución: 8 pasos por compás (corcheas) */

/* === SECCIÓN 3: DEFINICIÓN DE PATRONES === */
pattern kickPattern {
  /* Patrón de bombo: golpe en tiempos 1 y 5 (síncopa) */
  rhythm [x,.,.,.,x,.,.,.]
}

pattern snarePattern {
  /* Patrón de caja: golpe en tiempos 3 y 7 (backbeat) */
  rhythm [.,.,x,.,.,.,x,.]
}

pattern hihatPattern {
  /* Patrón de hi-hat: corcheas constantes */
  rhythm [x,x,x,x,x,x,x,x]
}

pattern bassline {
  /* Línea de bajo melódica: combina repetición y concatenación */
  /* Frase A (x2): E2 -> silencio -> A2 -> silencio */
  /* Frase B: E2 -> G2 -> A2 -> silencio */
  rhythm [E2,.,A2,.] * 2 + [E2,G2,A2,.]
}

/* === SECCIÓN 4: ASIGNACIÓN INSTRUMENTAL === */
instruments {
  kick {
    pattern kickPattern
    active 1-16          /* Activo durante todos los compases */
  }

  snare {
    pattern snarePattern
    active 1-16
  }

  hihat {
    pattern hihatPattern
  }
}

```



```

    active 5-16          /* Entra en el compás 5 */
}

bass {
    pattern bassline
    active 1-4 + 6-8 + 10-16 /* Rangos discontinuos en 5 y 9 */
}
}

```

Este programa genera un beat tipo drum'n'bass con bajo, sincronización entre ritmos y melodías, y rangos activos discontinuos que crean variación estructural.

Semántica Operacional

La semántica del lenguaje se define mediante transformaciones sobre el AST:

- Expansión de Repeticiones: $[e_1, \dots, e_n] * n \rightarrow [e_1, \dots, e_n, e_1, \dots, e_n, \dots, e_1, \dots, e_n]$ (n copias)
- Expansión de Concatenaciones: $\rho_1 + \rho_2 \rightarrow \text{append}(\text{expand}(\rho_1), \text{expand}(\rho_2))$
- Resolución de Importaciones: Para cada ``remember "file"`, se parsea "file" recursivamente, se detectan ciclos trackeando los archivos visitados y se mergean patrones importados al AST principal.`

3. Implementación

3.1. Frontend

El frontend del compilador implementa las dos primeras fases de compilación: análisis léxico y análisis sintáctico. Estas fases transforman el código fuente en un Árbol de Sintaxis Abstracta (AST) listo para análisis semántico y generación de código.

3.1.1. Análisis Léxico (Lexer)

El análisis léxico se implementa usando Flex 2.6.4 en modo PUSH (característica v2.0.0), que permite al lexer empujar tokens al parser de forma asíncrona en lugar del tradicional modo PULL. Archivo principal en `../FlexPatterns.l`. Y las acciones semánticas: en `../FlexActions.c`

Orden de patrones y modo PUSH:

El orden de las reglas en `FlexPatterns.l` es crítico para la correcta tokenización: primero se ubican comentarios, luego palabras clave, luego los tokens más específicos ("`x`", "`:`", notas) y al final el patrón de identificador general.

El lexer opera en modo PUSH: cada regla crea un Token con su valor semántico (entero o string) y lo envía explícitamente al parser, lo que permite desacoplar el ciclo Flex/Bison y manejar mejor errores y logs.

3.1.2. Análisis Sintáctico (Parser)

El análisis sintáctico se implementa usando "GNU Bison 3.8.2" con un parser LALR(1) que genera el AST completo. El AST implementa una arquitectura de nodos de datos + wrappers de lista:

Program agrupa ``imports``, ``declarations``, ``patterns`` e ``instruments``, mientras que los nodos de datos (``Pattern``, ``Instrument``, ``RhythmElement``, ``ActiveRange``) no contienen ``next``.

Las listas se representan con wrappers (`PatternList``, `InstrumentList``, `RhythmElementList``) que sí tienen ``next``, lo que simplifica el recorrido y la destrucción recursiva. Gracias a este formato se aseguró que cada producción gramatical tiene una acción semántica asociada que construye el nodo AST correspondiente.

Por último, el AST implementa más de 10 funciones destructoras que liberan recursivamente toda la memoria. Se ejecuta AddressSanitizer en todos los tests. No logramos detectar ningún caso con fugas de memoria en la versión final del proyecto.

3.2. Backend

El backend del compilador implementa las fases posteriores al frontend: análisis semántico y generación de código. En este compilador, el backend se enfoca en validación semántica exhaustiva y generación de archivos MIDI reproducibles.

3.2.1. Análisis Semántico

El análisis semántico valida que el programa es correcto según las reglas del dominio, más allá de la corrección sintáctica. Implementa tres componentes principales siguiendo la teoría descrita en los archivos provistos por la cátedra: ``Tabla de Símbolos``, ``Scopes``, y ``Sistema de Tipos``.

Tabla de Símbolos (.../SymbolTable.h y .c)

La tabla de símbolos es la "base de datos" del compilador, almacenando información sobre todos los patrones e instrumentos declarados.

La implementamos con una hash table dinámica.

Operaciones implementadas (siguiendo el archivo de Tabla de Símbolos):

Create (Inserción):

```
SymbolTable * createSymbolTable(int capacity) {...}
```

Read (Búsqueda):

```
Symbol * lookupSymbol(SymbolTable * table, const char * name) {...}
```

Update: No lo implementamos ya que el lenguaje no permite redeclaraciones.

Gestión de scopes:

El DSL de máquina de ritmos tiene un modelo de scopes simple:

- Scope global: Todos los patrones tienen ámbito global.
- No hay scopes anidados: Los instrumentos no definen nuevos scopes.
- No hay shadowing: No se permiten redeclaraciones.

Por lo tanto, no se requiere una pila de scopes. La tabla de símbolos única es suficiente.

Sistema de Tipos (...TypeChecker.h y .c)

Se implementan las funciones de verificación descritas, pero en el código se modelan como validadores booleanos (`true` = válido, `false` = error) en lugar de un enum de tipos explícitos.

Validaciones implementadas:

- Formato de notas: `validateNoteFormat` comprueba que la nota siga `[A-G][#b]?[0-9...]`, con letra A-G, alteración opcional y al menos un dígito de octava.
- Elementos y expresiones rítmicas: `validateRhythmElement/Array/Expression` aseguran que solo se usen `HIT`, `SILENCE` o `NOTE` válidas, que los arrays no estén vacíos y que las repeticiones tengan factor positivo.

- Rangos activos: ``validateActiveRange`` verifica que ``start`` y ``end`` sean positivos y ``start ≤ end``, mientras que ``validateActiveRangeWithinBounds`` exige además ``end ≤ compasses``.
- Declaraciones: ``validateDeclarations`` exige que ``tempo``, ``compasses`` y ``steps`` sean estrictamente positivos.

Analizador Semántico (.../SemanticAnalyzer.h y .c)

Orquesta la validación completa en 4 fases claramente separadas (ver ``SemanticAnalyzer.c``):

1. Validar declaraciones (si existen):

Se llama a ``validateDeclarations(program->declarations)`` y, si falla, se aborta el análisis.

2. Construir la tabla de símbolos:

``_buildSymbolTable(program)`` crea una ``SymbolTable`` con capacidad 64 e inserta todos los patrones e instrumentos mediante ``insertPattern`` / ``insertInstrument``. Detecta nombres duplicados.

3. Validar patrones:

``_validatePatterns(program)`` recorre la ``PatternList`` y utiliza ``validateRhythmExpression`` para asegurar que todas las expresiones rítmicas sean válidas.

4. Validar instrumentos:

``_validateInstruments(program)`` comprueba que:

- a. cada instrumento referencia un patrón existente (``lookupPattern`` en la tabla de símbolos global ``_symbolTable``);
- b. todos los rangos activos (``ActiveRange *activeRange`` y su lista encadenada) sean válidos y estén dentro de ``compasses`` (``validateActiveRange`` + ``validateActiveRangeWithinBounds``).

Los errores semánticos que pensamos que pueden ser los más comunes serían: patrones o instrumentos duplicados, referencias a patrones no definidos, rangos fuera de los límites, valor '0' para los rangos, y usar un formato de nota inválido.

3.2.2. Generación de Código MIDI

En los archivos `../MidiGenerator.h/c` y `../Generator.c`.`

La generación de código transforma el AST validado en un archivo MIDI binario reproducible, siguiendo los patrones descritos en los archivos provistos en la consigna por la cátedra: ``generacion de codigo` y `runtime`.`

Estrategia de Generación

Implementa el patrón de atributos sintetizados con descenso recursivo (LR):

1. Se visitar cada nodo desde la raíz.
2. Cada nodo delega la generación a sus hijos.
3. Los resultados de los hijos se combinan para generar la salida del nodo actual.
4. Función ``output()`` encapsula la escritura binaria.

Implementación del header y eventos (resumen):

Para formar y generar el archivo MIDI válido hubo que tener que tomar muchas consideraciones específicas a la formación de estos archivos, algunas fueron:

- El header ``MThd`` se escribe en **big-endian**, con tamaño fijo 6 bytes, formato 1 y ``numTracks = 1 + número de instrumentos`.`

- Los delta times se codifican como **Variable-Length Quantity** (VLQ), empaquetando grupos de 7 bits con un bit de continuación; esta lógica está centralizada en una función auxiliar (ver `MidiGenerator.c`).
- Los eventos Note On/Off se generan escribiendo: delta time (VLQ), status byte (`'0x9?' / '0x8?' + canal`) y dos data bytes (nota, velocity).

Mapeo de Elementos Rítmicos a MIDI

- `'HIT'` se mapea a canal 10 (índice 9) y nota 36 (Bass Drum 1, General MIDI).
- `'SILENCE'` no genera eventos.
- `'NOTE'` se mapea a canal melódico (canal 1) y número de nota MIDI calculado desde notación anglosajona (`'[A-G][#b]?[0-9...]'`).

Cálculo de Timing

El timing preciso es crítico para la reproducción correcta.

El compilador fija `'TPQN = 480'` (ticks por negra) y toma cada `'step'` como una negra completa. Ticks por compás: `'ticksPerCompass = TPQN * steps'`. Tick absoluto de un evento: `'((compás - 1) * ticksPerCompass) + (índiceStep * ticksPerStep)'`.

Los detalles finos de implementación (incluyendo los bugs de timing y distribución de elementos) se documentan en la sección 3.4. Dificultades Encontradas.

Separación Compiler-Runtime (siguiendo `'runtime'`)

El compilador y el runtime están completamente desacoplados:

El Compiler asume todas las responsabilidades internas del flujo: parseo del código DSL, validación semántica y generación del fichero MIDI; por su parte, las tareas relacionadas con la reproducción y la síntesis de audio se delegan al Client, es decir, al reproductor MIDI del usuario y al soundfont o motor de síntesis que éste utilice.

Las ventajas de esta separación son la portabilidad, dado que el archivo MIDI (`.mid`) funciona en cualquier sistema con reproductor MIDI (Windows Media Player, VLC, GarageBand, timidity, etc.). Además, no se requieren instalar bibliotecas de audio, gracias a que el proyecto tiene todo integrado. Gracias a la versatilidad que ofrece, el usuario puede elegir su reproductor y soundfont preferidos. Por último, MIDI es un formato abierto con 40+ años de soporte, lo que garantiza confiabilidad a la hora de hacer la conversión.

En cuanto a desventajas, notamos una importante en la generación de MIDI: El programa no controla cómo se sintetizan los sonidos, se depende de los soundfonts del cliente, cuya calidad es desconocida y puede empeorar la experiencia, o causar que distintos usuarios escuchen cosas distintas.

Sin embargo, esto fue solucionado con la generación del .WAV, que permite que todos los usuarios escuchen de la misma forma el mismo input. Al dejar también la generación de MIDI le damos a los usuarios la posibilidad de usarlo con los soundfonts que desee.

3.3. Adicionales

3.3.1. Generación y Conversión de Output de Audio

Se realiza en los archivos “.../code-generation/ Generator.h y .c”, .../output-conversion/ OutputConversion.h y .c y se orquesta desde EntryPoint.c.

Introduce una fase luego de la generación de código en el backend. Primero se genera el archivo MIDI a partir del AST, y luego, de forma opcional, convierte ese MIDI a uno o más formatos de audio (WAV, FLAC, MP3) según flags de línea de comandos y la variable de entorno ‘OUTPUT_CONVERSION_ENABLED’. Así se logra generar más formatos de salida útiles sin mezclar las responsabilidades entre generación de código (MIDI) y conversión de salida (audio final).

En términos generales, el flujo funciona de la siguiente manera:

1. En EntryPoint.c se amplía el CompilerState con tres flags: outputWav, outputFlac, outputMp3.
2. Al iniciar main, se parsea el argumento -fo=wav,flac,mp3:
 - Se recorre argv[] buscando argumentos que comiencen con -fo=.
 - Se parsea la lista separada por comas y se activan los flags correspondientes en CompilerState (por ejemplo, -fo=wav,mp3 activa solo outputWav y outputMp3).
3. Finalizada la generación de código, EntryPoint.c invoca una nueva etapa executeOutputConversion(&compilerState) definida en OutputConversion.c.
4. El módulo OutputConversion decide si ejecutar o no la conversión:
 - Si OUTPUT_CONVERSION_ENABLED no está definida o no es "true", la etapa se salta por completo (solo queda el .mid).
 - Si está habilitada y al menos uno de los flags outputWav/outputFlac/outputMp3 está activo, se procede a la conversión.

3.3.2. Sistema de Importación Recursiva

Se realiza en el archivo `src/main/c/frontend/ImportResolver.h/c`.

Implementa la directiva `remember "file.dsl"` con las capacidades de Carga recursiva de archivos, detección de dependencias circulares, merging de patrones al AST principal y manejo de rutas relativas

Implementación:

En términos generales, el algoritmo funciona de la siguiente manera:

1. Mantiene una pila de archivos visitados (`\_visitedFiles`, `\_visitedCount`) para detectar ciclos y respetar una profundidad máxima de imports.
2. Para cada `remember "file.dsl"`:
 - Verifica existencia del archivo.

- Crea un lexer/parser independientes y parsea el archivo completo a un ``Program * importedProgram``.

3. Resuelve recursivamente los imports anidados de ``importedProgram``.

4. Hace merge de la ``PatternList`` importada dentro del programa principal (``Program * mainProgram``), transfiriendo el ownership de los patrones.

5. Destruye el AST importado y hace backtracking en la pila de visitados para permitir patrones en diamante sin falsos positivos de ciclo.

Detección de ciclos con backtracking:

El algoritmo mantiene una pila de archivos visitados (``_visitedFiles[]``) y usa backtracking para permitir patrones de importación en 'diamante' sin dejar bloquear los ciclos.

Gestión de memoria con múltiples parsers (Bug #5):

Cada archivo importado requiere su propio lexer. Esto causó fugas de memoria porque ``initializeFlexActionsModule()`` creaba un nuevo logger en cada invocación.

3.3.3. Sistema de Logging Configurable

El compilador hace uso del sistema de logging provisto. ALL, DEBUGGING, INFORMATION, WARNING, ERROR, CRITICAL.

Mediante la variable de entorno LOGGING_LEVEL se puede configurar el nivel de logging que se desea ver.

3.3.4. Verificación de Memoria con AddressSanitizer

Todos los tests ejecutados a lo largo del proyecto fueron ejecutados con AddressSanitizer para detectar memory leaks, use-after-frees, buffer overflow, double freeing y otros problemas de memoria que se puedan tener en c.

Esto está declarado dentro del CMakeFile.

```
CFLAGS += -fsanitize=address -fno-omit-frame-pointer -g
LDLAGS += -fsanitize=address
```

3.4. Dificultades Encontradas

Durante el desarrollo del compilador, se encontraron 5 bugs significativos que requirieron análisis profundo y refactoring. Cada bug proporcionó lecciones valiosas sobre compiladores, MIDI, y gestión de memoria.

- Bug #1: Orden de Elementos Rítmicos (Melodía Invertida)

Notamos que las melodías se reproducían al revés ([E2,G2,A2]` sonaba como `A2,G2,E2`). Esto ocurría gracias a la acción semántica que construía `RhythmElementList` hacía “prepending” en lugar de “appending”, invirtiendo el orden natural de los elementos al reducir `rhythm_element_list: rhythm_element_list COMMA rhythm_element`. Para solucionarlo se tuvo que cambiar la acción para recorrer la lista hasta el final y “appendar” el nuevo nodo, preservando el orden de aparición en el código fuente.

- Bug #2: Distribución de Elementos en Compases

Al realizar las primeras pruebas, vimos que todos los elementos del patrón se reproducían en cada compás, generando un efecto de “muro de sonido” y aceleración artificial. Esto era debido al bucle de generación MIDI recorría la lista expandida completa para cada compás, en lugar de tomar solo el “slice” correspondiente a ese

compás. Fue necesario calcular cuántos elementos caben por compás (`steps`), dividir el patrón expandido en chunks y, para cada compás, reproducir únicamente el chunk adecuado, ciclando con módulo cuando el instrumento está activo más compases que los que ocupa el patrón.

- Bug #3: Timing MIDI (Reproducción 4x Más Rápida)

Observamos que los archivos MIDI se reproducían 4 veces más rápido de lo esperado. Un programa con `tempo 120` sonaba como tempo 480, gracias a que se interpretaba erróneamente que cada `step` era una fracción de negra, y se calculó $\text{ticksPerStep} = \text{TPQN} / \text{steps}$. En el diseño final del lenguaje, cada `step` es una negra completa, por lo que debía usarse $\text{ticksPerStep} = \text{TPQN}$. Para evitar que siga sucediendo, hubo que fijar $\text{ticksPerStep} = \text{TPQN}$ y derivar a partir de allí los ticks por compás y los tiempos absolutos. El bug mostró lo delicado que es interpretar correctamente la especificación MIDI.

- Bug #4: Declaraciones Obligatorias en Archivos de Biblioteca

Al realizar los tests CI/CD, éstos fallaban al intentar parsear archivos de biblioteca (pattern-only files) que no contenían declaraciones (`tempo`, `compasses`, `steps`). La causa de este bug radicaba en el analizador semántico, que asumía que `program->declarations` nunca era `NULL`, ignorando que la gramática permite archivos con solo patrones (bibliotecas). Fue necesario hacer que `analyzeProgram` trate las declaraciones como opcionales (validarlas solo si existen) y que tanto la validación de rangos activos como la generación de MIDI se salten cuando el archivo es solo una librería de patrones. Esto respeta el modelo de `*scope global*` para patrones.

- Bug #5: Fugas de Memoria Durante Importación

Vimos que el AddressSanitizer reportaba pequeñas fugas de memoria al ejecutar tests que usaban imports encadenados. El problema es que cada import inicializaba nuevamente los módulos de Flex/Bison, creando loggers adicionales que nunca se destruían.

Para solucionarlo, decidimos aplicar un patrón de inicialización única: “initializeFlexActionsModule” y “initializeBisonActionsModule”. Ahora se crea un logger solo si “_logger == NULL”, compartiendo el estado entre todas las instancias de lexer/parser de imports. Esto arregló las fugas con la menor complejidad posible.

4. Futuras Extensiones

A futuro, se identificaron posibles líneas de trabajo, agrupadas en cuatro ejes:

4.1 Extensiones del lenguaje

- Velocidad variable por evento.
- Selección explícita de instrumentos y canales MIDI por instrumento.
- Duración de notas configurable, más allá de 1 step fijo.
- Azúcar sintáctico (constantes, operadores adicionales como interleaving).

4.2 Extensiones de análisis

- Advertencias de optimización (repeticiones innecesarias, patrones silenciosos, etc.).
- Detección de polirritmias y patrones que no calzan exactamente en la grilla de compases.

4.3 Extensiones del backend

- Exportar a otros formatos (JSON, MusicXML) reutilizando el AST expandido.

4.4 Herramientas y performance

- Visualizador de patrones (ASCII/HTML) para debugging rápido.
- REPL interactivo orientado a experimentación.
- Compilación incremental y posibles optimizaciones de rendimiento (paralelización, MIDI más compacto).

5. Conclusiones

En este Stage final, se implementó exitosamente un compilador completo para un Lenguaje Específico de Dominio (DSL) orientado a la composición de patrones rítmicos para máquinas de ritmos. El compilador transforma código declarativo de alto nivel en archivos MIDI estándar reproducibles, demostrando los principios fundamentales de la teoría de compiladores aplicados a un dominio concreto.

5.1. Lecciones Aprendidas

Durante el desarrollo de este último Stage, aprendimos que el orden de las reglas en Flex es determinante; los patrones específicos (comentarios, keywords, ``x``, ``.``, notas) deben ir antes que el identificador genérico para evitar clasificaciones erróneas. Además observamos que aún con destructores cuidadosos, es fácil introducir fugas o “use-after-free”. Es por esto que usar sanitizers desde el inicio simplifica enormemente el debugging. Por otro lado, los bugs más sutiles (melodías invertidas, timing incorrecto, leaks en imports) aparecieron solo gracias a una combinación de tests de aceptación/rechazo, escuchar el MIDI y ejecutar AddressSanitizer. También aprendimos que el hecho de implementar MIDI obligó a respetar al detalle TPQN (Ticks Per Quarter Note), y VLQ(Variable Length Quantity), por lo tanto, fue necesario realizar un estudio más extenso. Finalmente, generar solo MIDI y delegar la síntesis al entorno del usuario dio como resultado un compilador más portable, fácil de probar y sencillo de integrar con distintas herramientas.

5.2. Reflexión Final

La realización de este Stage III, nos ayudó a comprender la idea de que los compiladores son programas complejos que transforman representaciones abstractas. El salto conceptual de parsear texto a generar archivos binarios reproducibles requiere entender muy bien la teoría de lenguajes formales (autómatas, gramáticas), estructuras de datos como árboles, tablas hash, o listas enlazadas. Por

otro lado, el conocimiento de algoritmos tales como “traversal recursivo” o “hashing” nos fueron de utilidad para recorrer el AST, y detectar duplicados rápidamente. En nuestro caso, tuvimos que leer mucha documentación sobre el uso de MIDI, era importante para poder convertir el output del compilador, en un archivo reproducible. Por último, la etapa de testing, aunque algo molesta, es necesaria para reducir cualquier probabilidad de error al parsear, y “arrastrar” dicho bug hasta el output. El resultado de todos estos conocimientos implementados es un compilador robusto y extensible que cumple su objetivo de transformar texto en música.

6. Referencias

Para la realización de este trabajo, se utilizó como referencia las fuentes:

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
- Levine, J. (2009). *flex & bison: Text Processing Tools*. O'Reilly Media.
- Paxson, V., & Estes, W. (2020). *Flex - The Fast Lexical Analyzer* (Version 2.6.4). Free Software Foundation.
- Donnelly, C., & Stallman, R. (2020). *Bison - The GNU Parser Generator* (Version 3.8.2). Free Software Foundation.
- MIDI Manufacturers Association. (1996). *The Complete MIDI 1.0 Detailed Specification* (Document Version 96.1). MIDI Manufacturers Association.
- Music Technology Group, McGill University. *Standard MIDI File Format*. McGill University.

7. Bibliografía

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Cooper, K. D., & Torczon, L. (2011). *Engineering a Compiler* (2nd ed.). Morgan Kaufmann.